



The National University of Computer and  
Emerging Sciences

# Decision Trees

## Machine Learning for Data Science

**Dr. Akhtar Jamil**

**Department of Computer Science**

# Goals

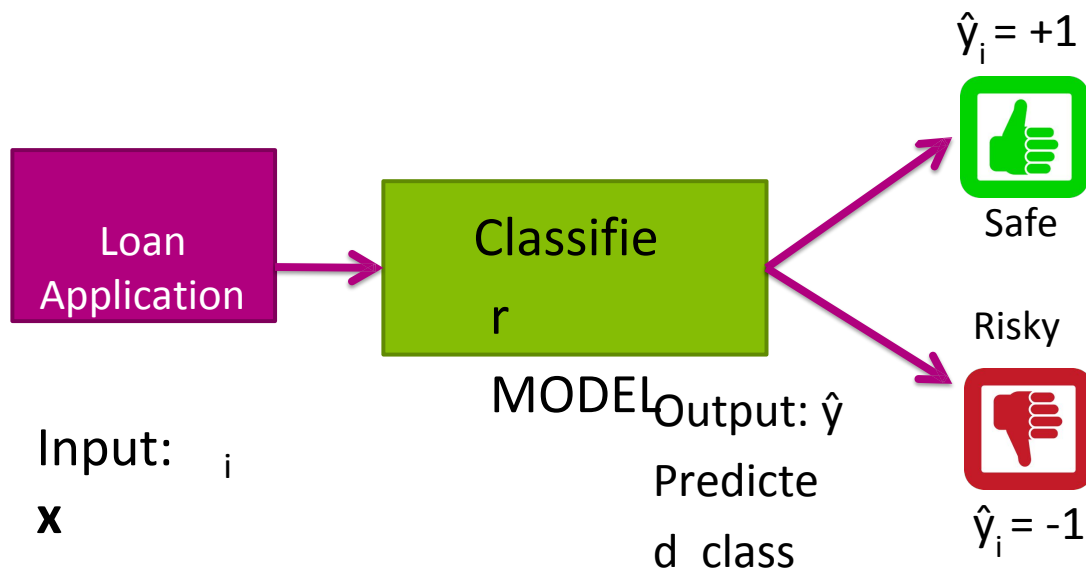
- Today's Lecture
  - Decision Trees

# Motivation

- Problem: Given the following information of a customer, decide whether the bank should assign a loan or not
- Classes: Safe or Risky
- Features: Credit History, Income, Term, Personal Info

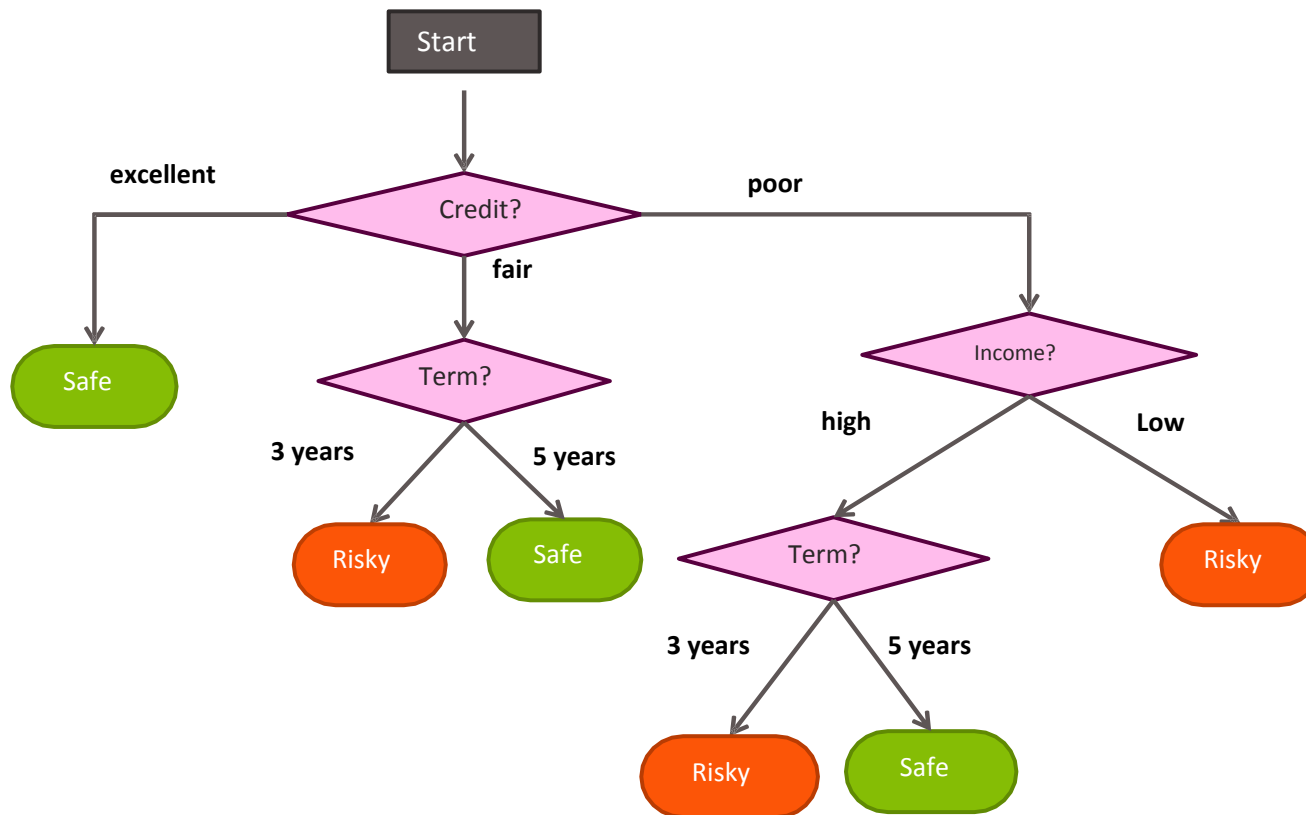
| Credit    | Term  | Income | y     |
|-----------|-------|--------|-------|
| excellent | 3 yrs | high   | safe  |
| fair      | 5 yrs | low    | risky |
| fair      | 3 yrs | high   | safe  |
| poor      | 5 yrs | high   | risky |
| excellent | 3 yrs | low    | risky |
| fair      | 5 yrs | low    | safe  |
| poor      | 3 yrs | high   | risky |
| poor      | 5 yrs | low    | safe  |
| fair      | 3 yrs | high   | safe  |

# Motivation



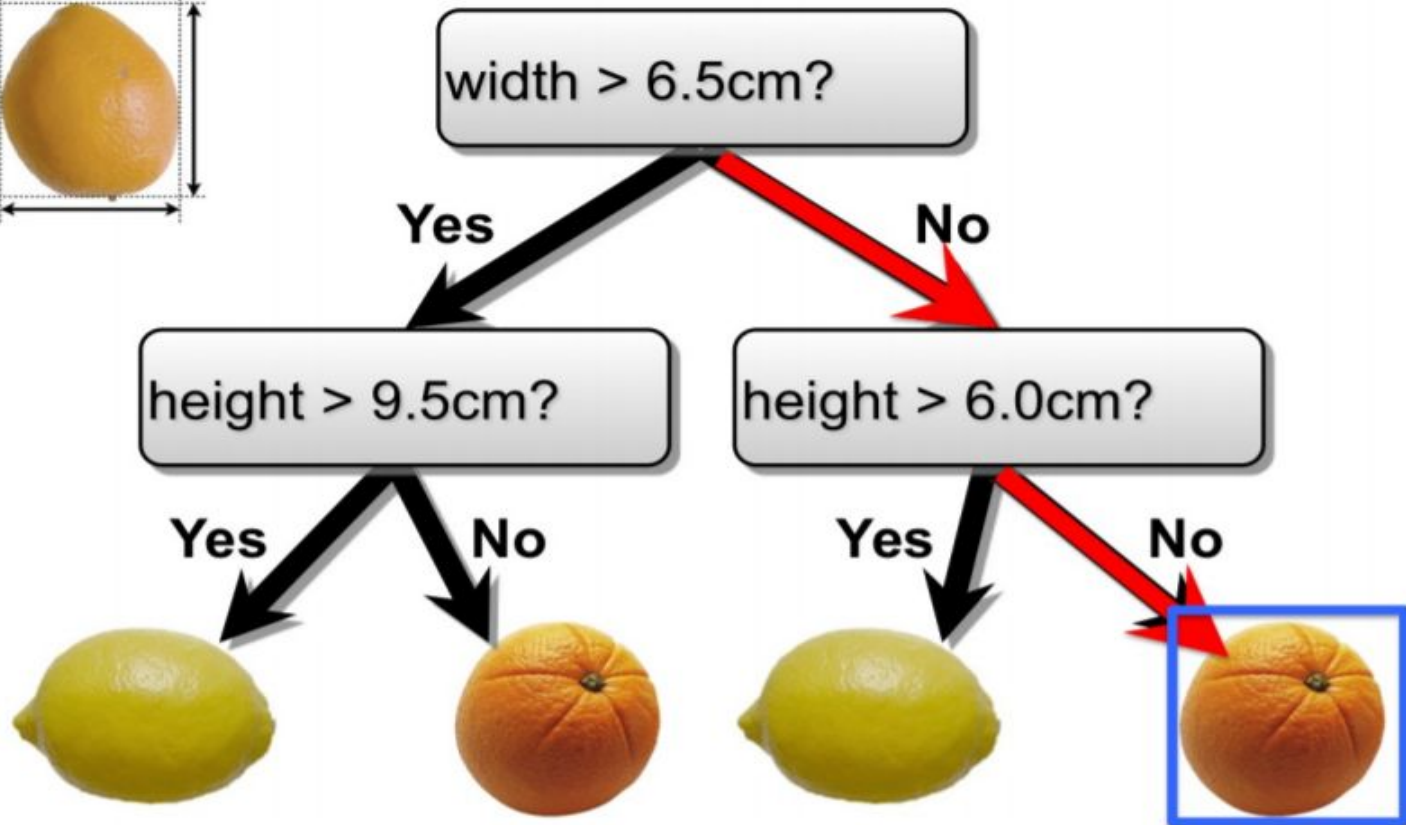
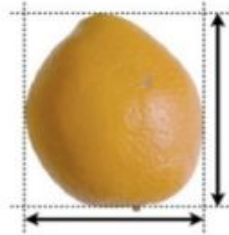
# Motivation

$x_i = (\text{Credit History} = \text{poor}, \text{Income} = \text{high}, \text{Term} = 5 \text{ years})$

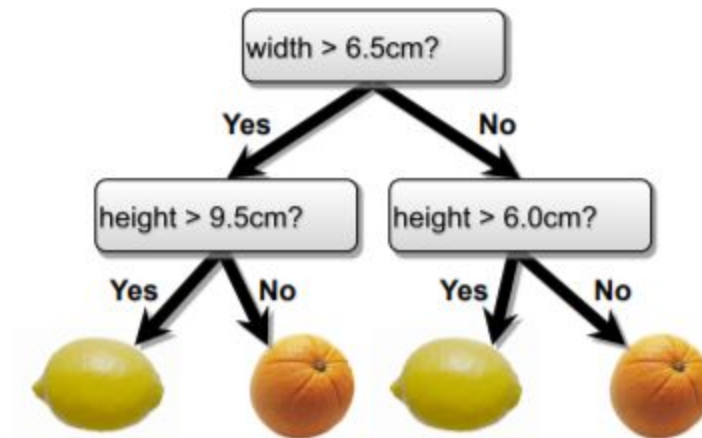


# Motivation

Test example



# Motivation



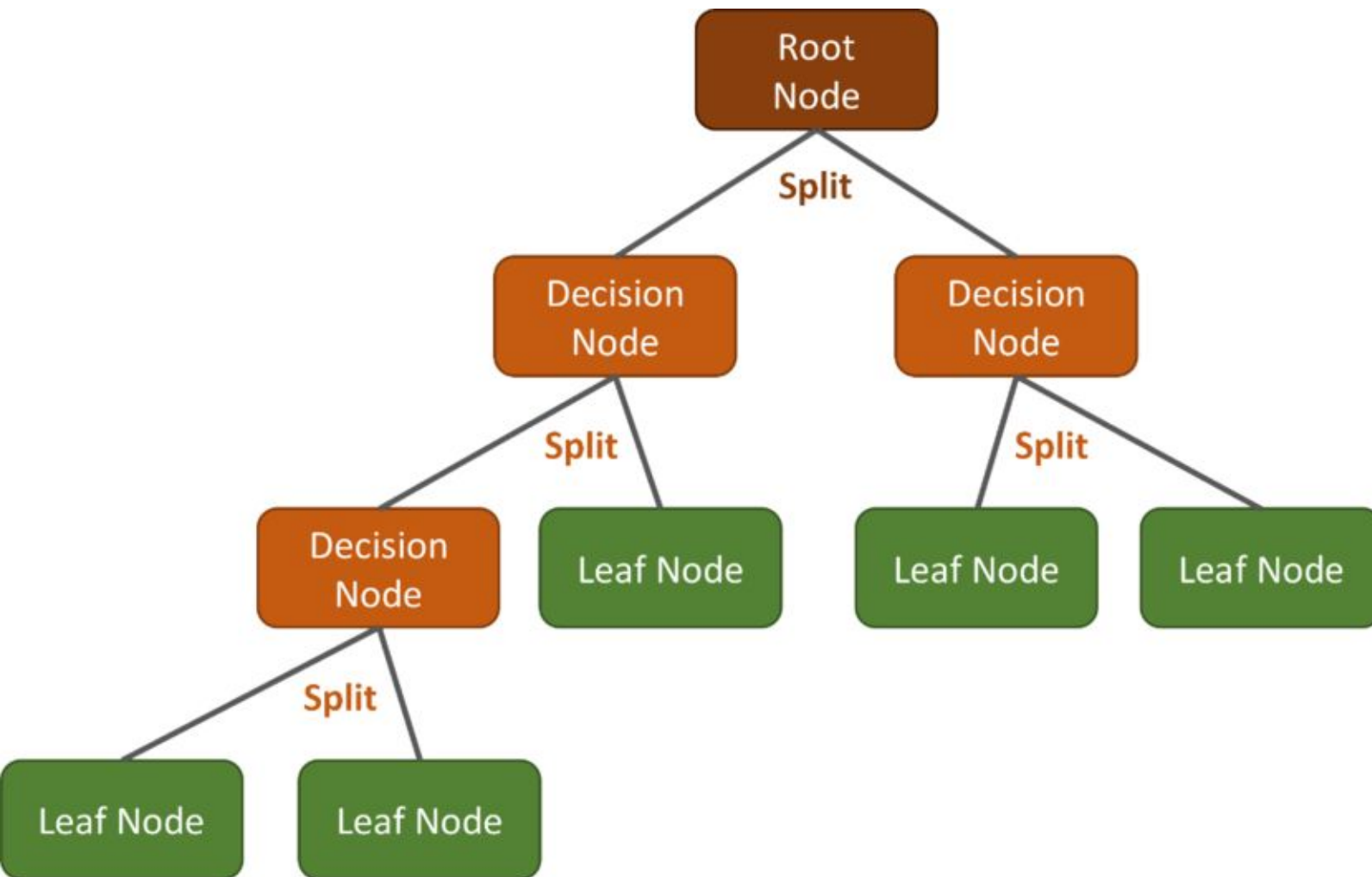
- Internal nodes **test attributes**
- Branching is determined by **attribute value**
- Leaf nodes are **outputs** (class assignments)

# Decision Trees

- **Decision tree (DT)** is a machine learning algorithm.
- Its learning function is represented by a **decision tree**.
- DT can also be represented as sets of **if-then rules**.
- Among the most popular of **inductive inference algorithms**
- Applied to a broad range of tasks:
  - Medical diagnosis
  - Credit risk analysis
  - Object classification



# Decision Trees



# Classification and Regression Trees (CART)

- How do we organize the tree to **produce the lowest classification error?**
- Any decision tree will successively **split the data into smaller and smaller subsets.**
  - It would be ideal if all the samples associated with a leaf node were from the same class.
  - Such a subset, or node, is considered **pure** in this case.
- A generic tree-growing methodology, known as CART, successively splits nodes until they are pure.
  - **Classification and Regression Tree**

# Classification and Regression Trees (CART)

- Six key questions:

- 1) Should the questions be binary (e.g., is gender male or female) or numeric (e.g., is height  $\geq 5'4''$ ) or multi-valued (e.g., race)?
- 2) Which feature should be tested at each node?
- 3) When should a node be declared a leaf?
- 4) If the tree becomes too large, how can it be pruned?
- 5) If the leaf node is impure, what category should be assigned to it?
- 6) How should missing data be handled?

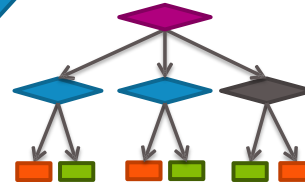
# Decision tree learning problem

Training data:  $N$  observations  
 $(x_i, y_i)$

| Credit    | Term  | Income | y     |
|-----------|-------|--------|-------|
| excellent | 3 yrs | high   | safe  |
| fair      | 5 yrs | low    | risky |
| fair      | 3 yrs | high   | safe  |
| poor      | 5 yrs | high   | risky |
| excellent | 3 yrs | low    | risky |
| fair      | 5 yrs | low    | safe  |
| poor      | 3 yrs | high   | risky |
| poor      | 5 yrs | low    | safe  |
| fair      | 3 yrs | high   | safe  |

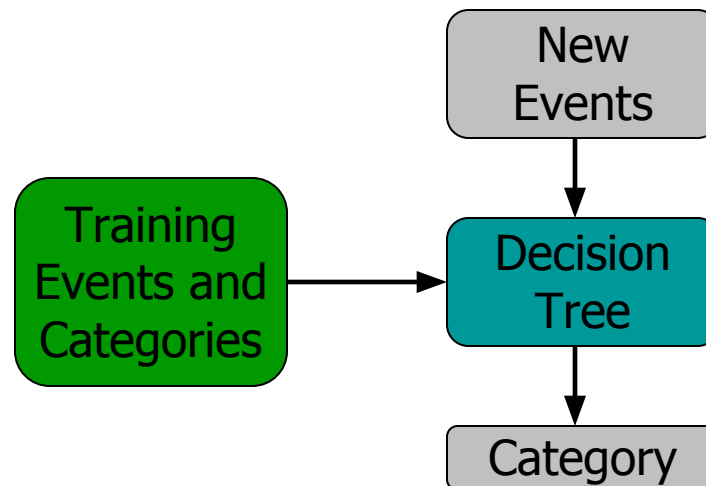
Optimize  
quality metric  
on training data

$T(X)$



# Decision Trees

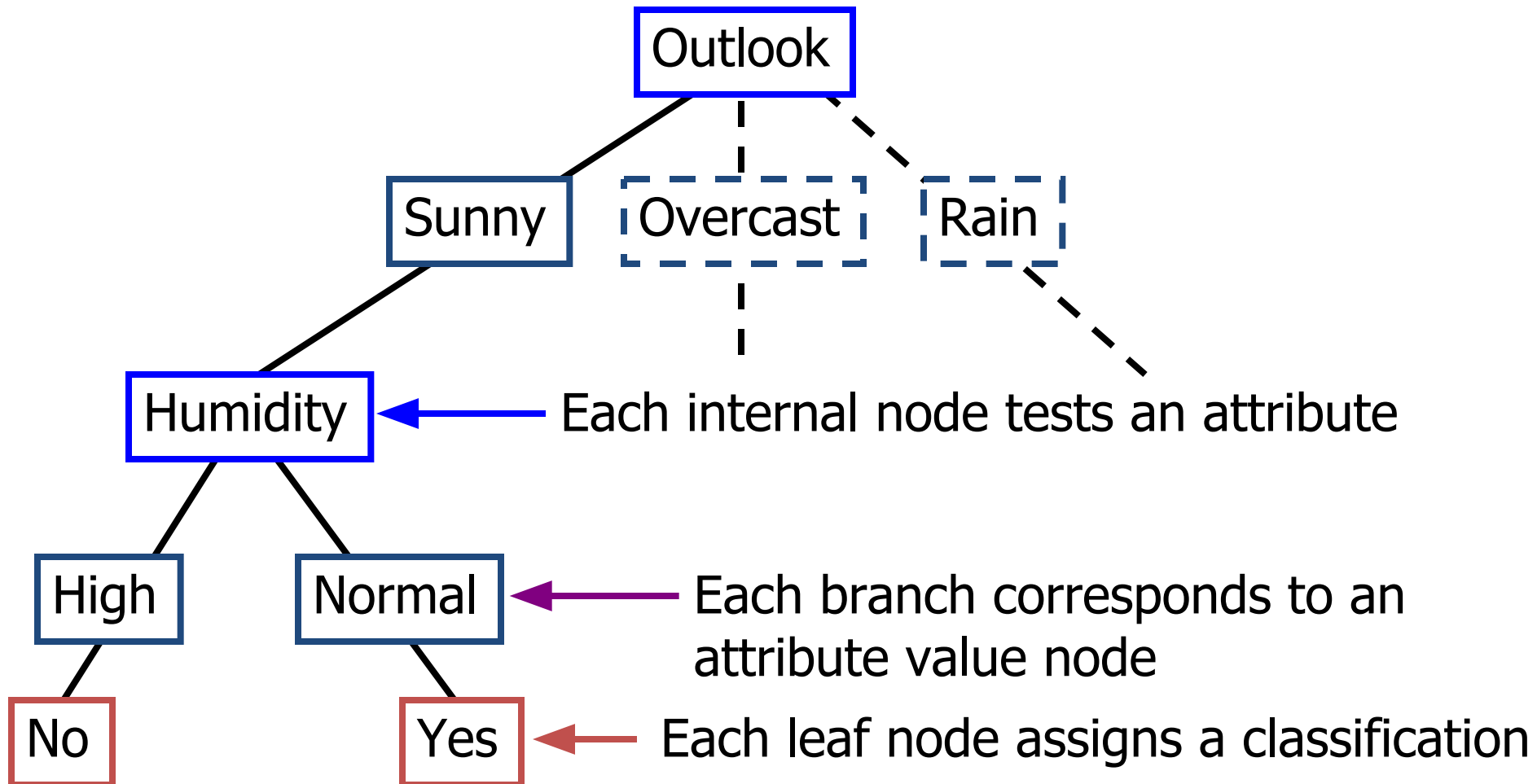
- Use a decision tree to **predict categories for new events**.
- Use **training data** to build the decision tree.



# Classification and Regression

- Each path from root to a leaf defines a region  $R_m$  of input space
- Let  $\{(x^{(m_1)}, t^{(m_1)}), \dots, (x^{(m_k)}, t^{(m_k)})\}$  be the training examples that fall into  $R_m$
- **Classification tree:**
  - ▶ discrete output
  - ▶ leaf value  $y^m$  typically set to the most common value in  $\{t^{(m_1)}, \dots, t^{(m_k)}\}$
- **Regression tree:**
  - ▶ continuous output
  - ▶ leaf value  $y^m$  typically set to the mean value in  $\{t^{(m_1)}, \dots, t^{(m_k)}\}$

# Decision Tree Representation



# Decision Tree Algorithm

---

- The basic algorithm of decision tree employs **top-down and greedy search**
- The main algorithm that we will discuss is **ID3** developed by **Ross Quinlan (1986)**



# ID3 Algorithm

---

- **Iterative Dichotomiser**
  1.  $A \leftarrow$  select the "best" decision attribute to split the examples
  2. For each value of A create new child nodes
  3. Split training examples according to the attribute value (Sunny, Overcast, Rain)
  4. If all training examples are perfectly classified stop, else iterate over new leaf nodes.

# Quality metric: Classification error

- Error measures fraction of mistakes

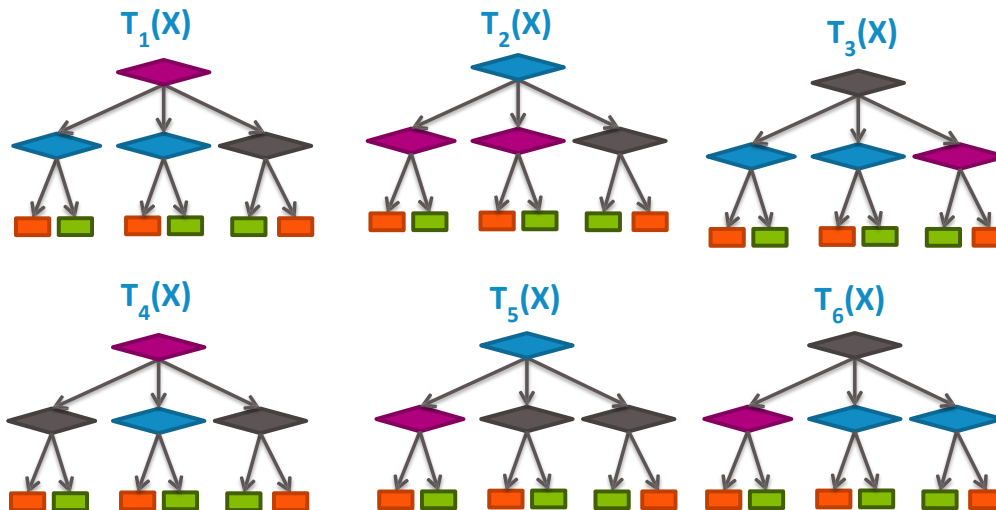
$$\text{Error} = \frac{\text{\# incorrect predictions}}{\text{\# examples}}$$

- Best possible value : 0.0
- Worst possible value: 1.0

# How do we find the best tree?

Exponentially large number of possible trees makes decision tree learning **hard**!

Learning the smallest decision tree is an *NP-hard problem*  
[Hyafil & Rivest '76]



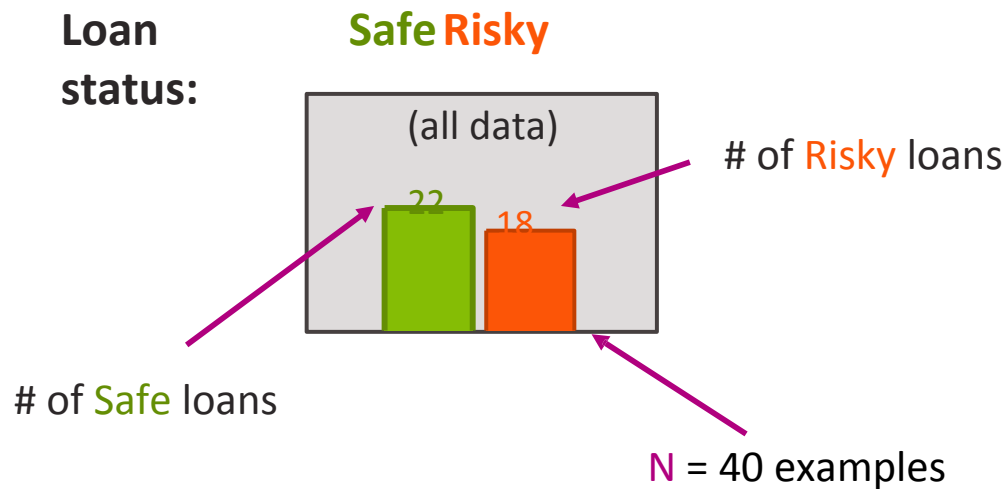
# Our training data table

Assume  $N = 40, 3$   
features

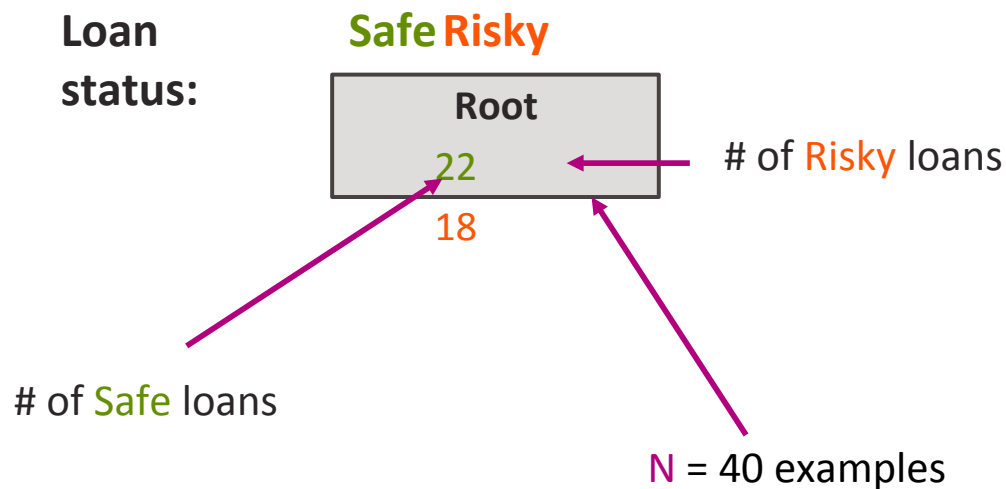
| Credit    | Term  | Income | y     |
|-----------|-------|--------|-------|
| excellent | 3 yrs | high   | safe  |
| fair      | 5 yrs | low    | risky |
| fair      | 3 yrs | high   | safe  |
| poor      | 5 yrs | high   | risky |
| excellent | 3 yrs | low    | risky |
| fair      | 5 yrs | low    | safe  |
| poor      | 3 yrs | high   | risky |
| poor      | 5 yrs | low    | safe  |
| fair      | 3 yrs | high   | safe  |

## Greedy decision tree learning

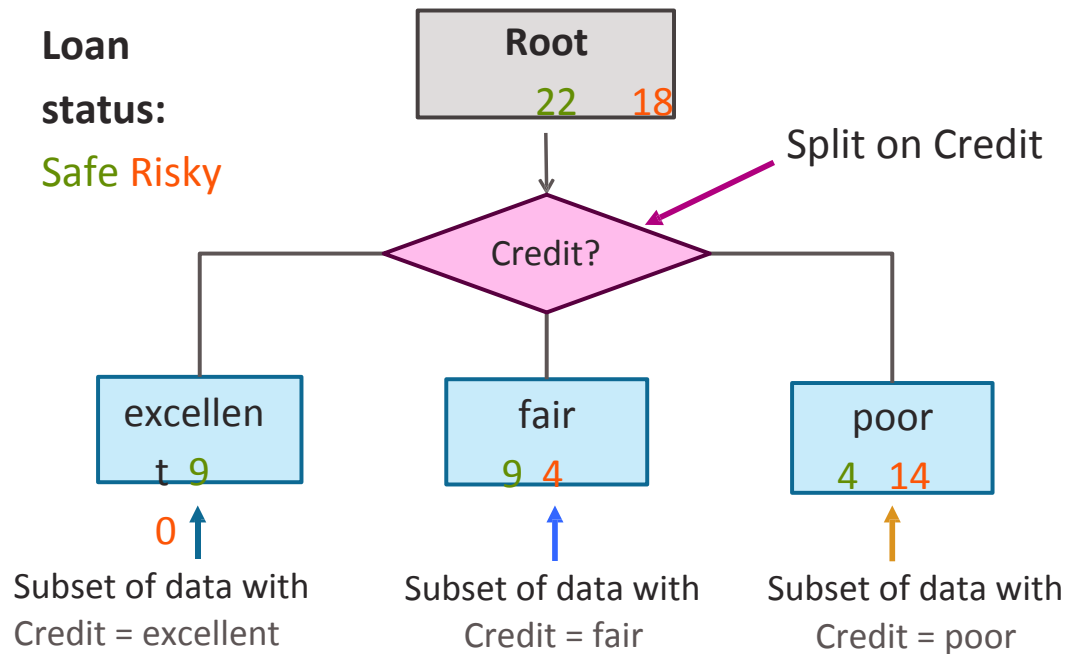
# Start with all the data



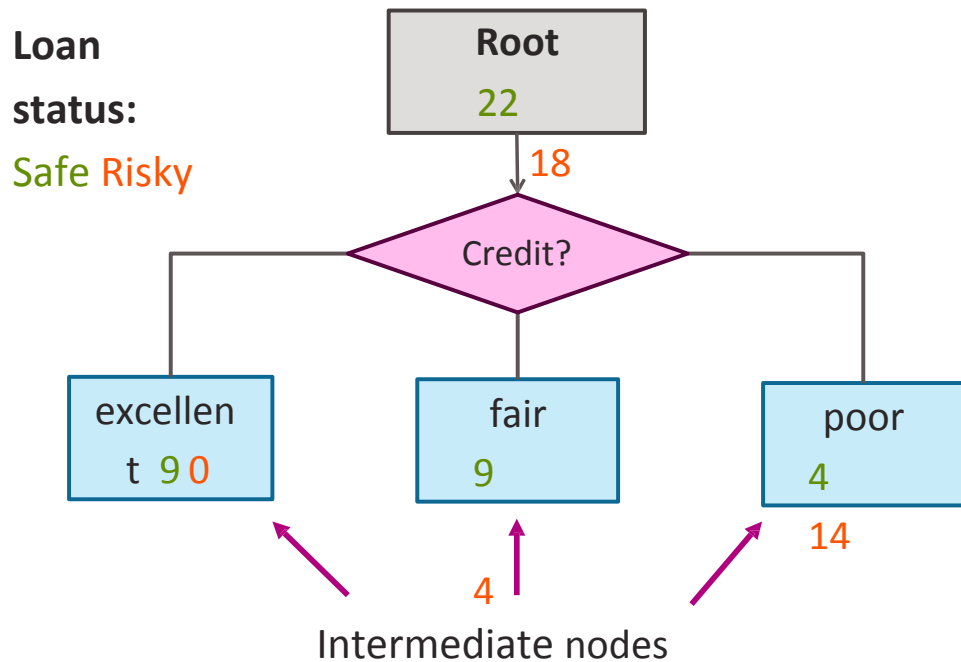
# Compact visual notation: Root node



# Decision stump: Single level tree

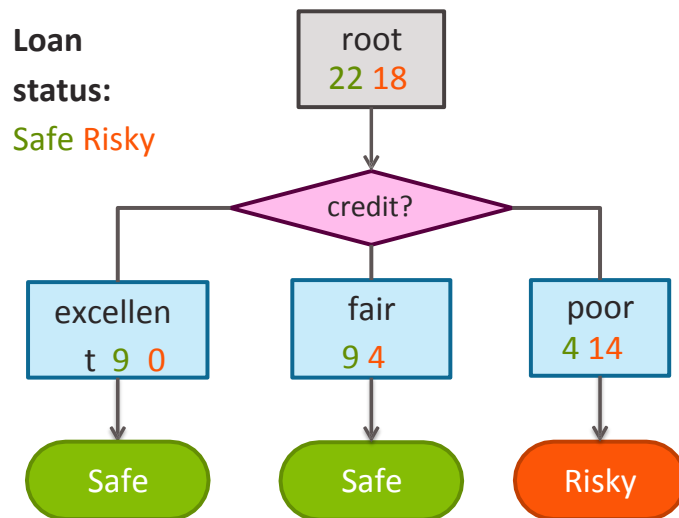


# Visual notation: Intermediate nodes





# Making predictions with a decision

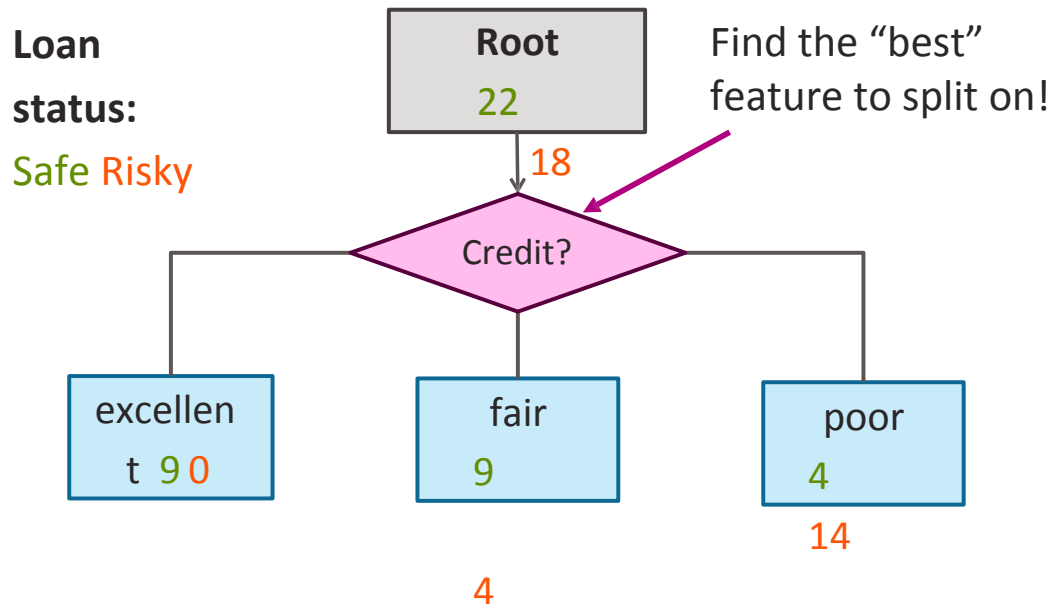


For each intermediate node, set  $\hat{y}$  = majority value

---

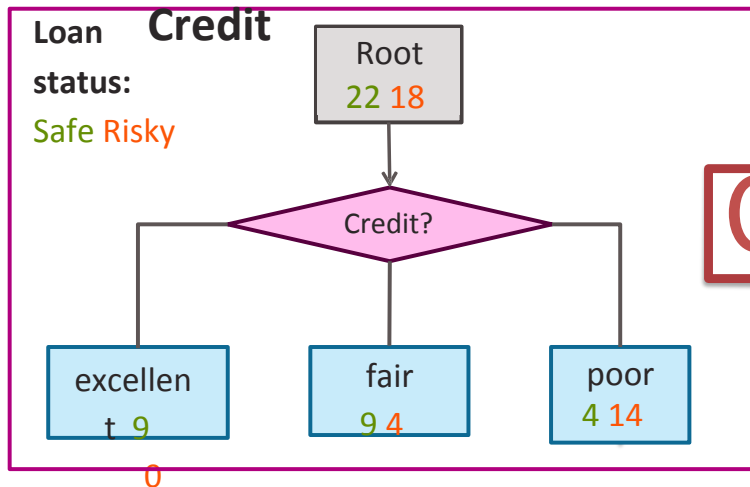
## Selecting best feature to split on

# How do we learn a decision stump?



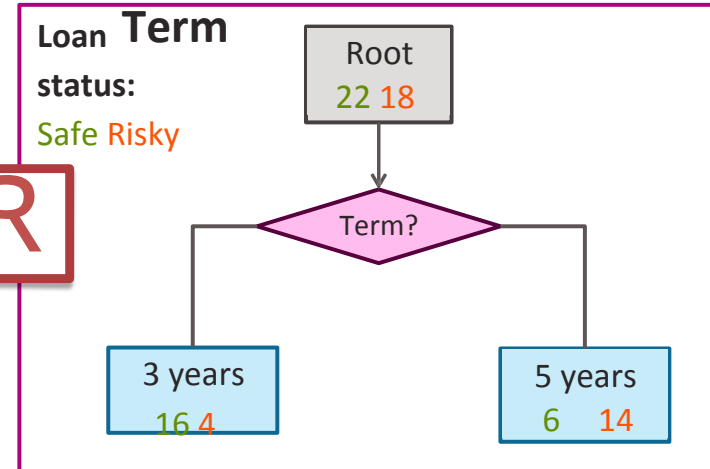
# How do we select the best feature?

## Choice 1: Split on

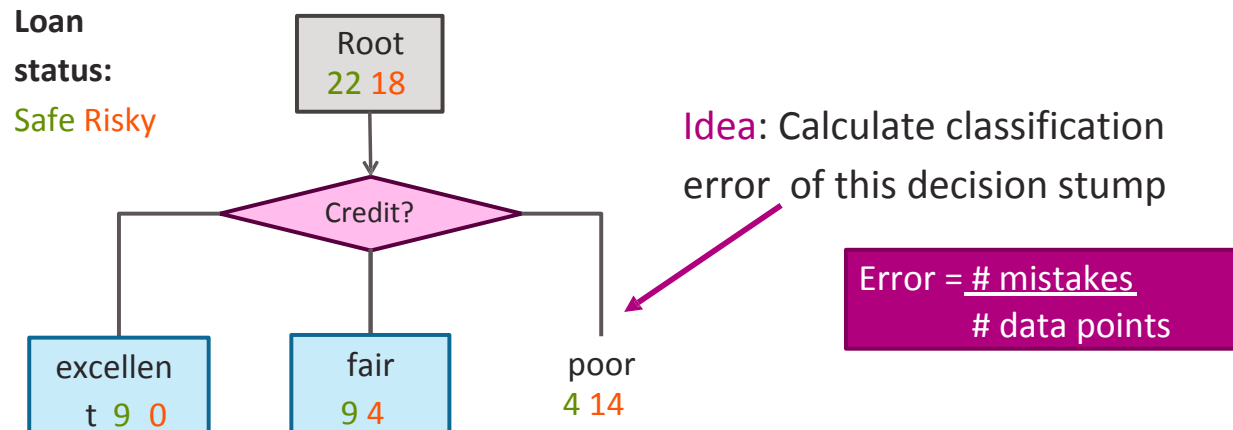


OR

## Choice 2: Split on

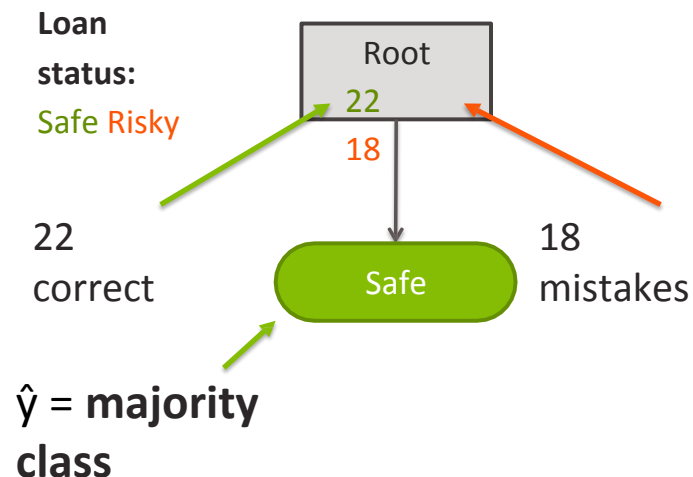


# How do we measure effectiveness of a split?



# Calculating classification error

- **Step 1:**  $\hat{y}$  = class of majority of data in node
- **Step 2:** Calculate classification error of predicting  $\hat{y}$  for this data

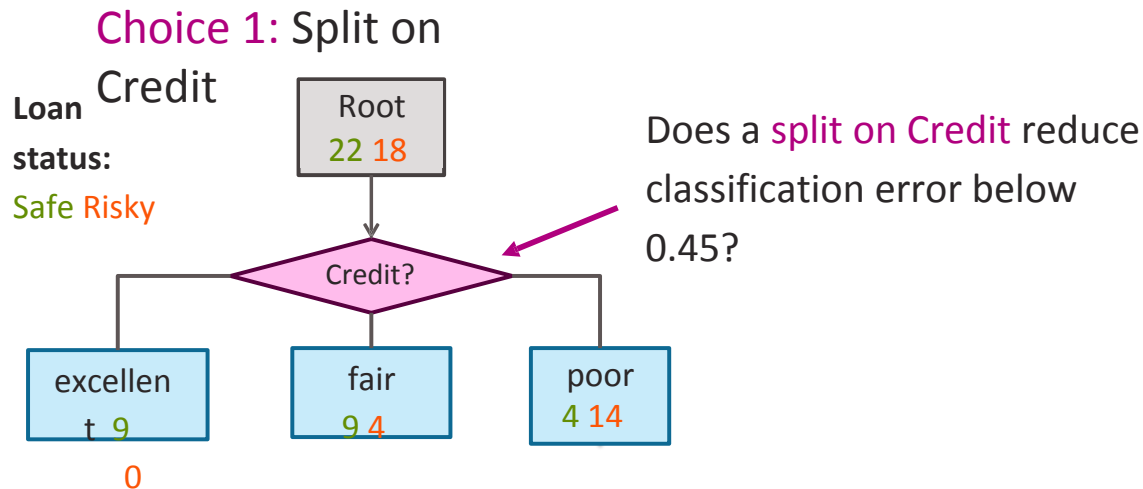


Error =

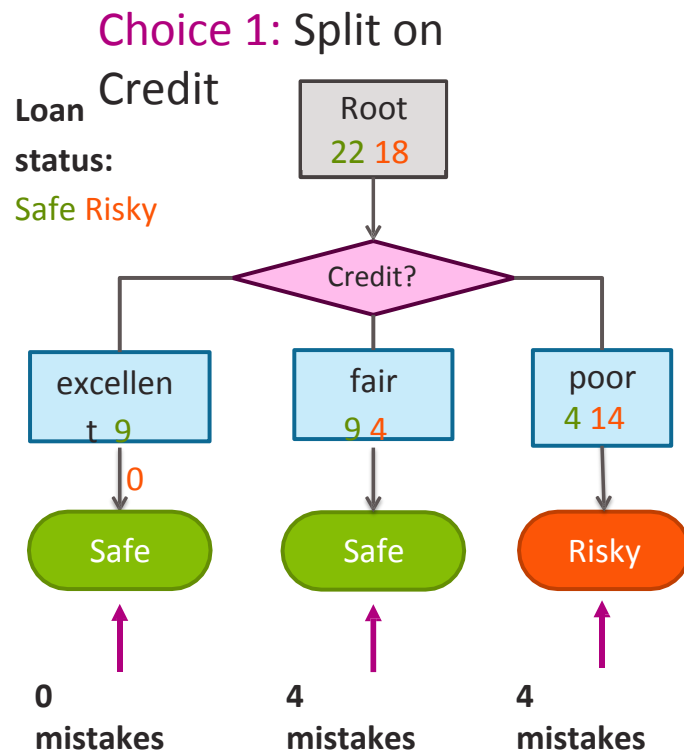
=

| Tree   | Classification error |
|--------|----------------------|
| (root) | 0.45                 |

# Choice 1: Split on Credit history?



# Split on Credit: Classification error



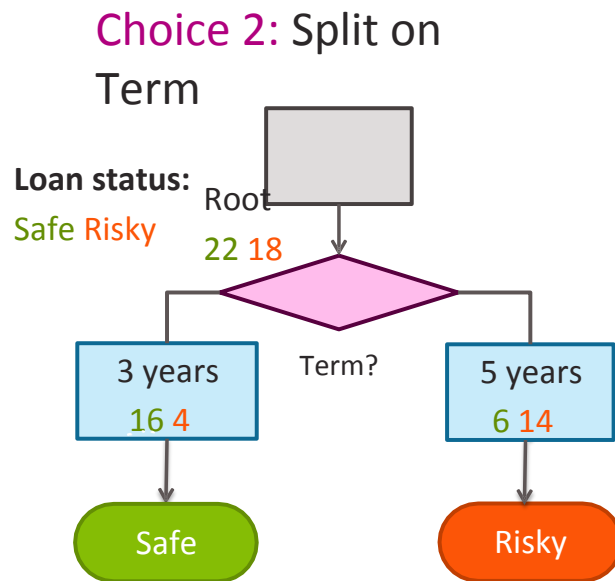
Error =

=

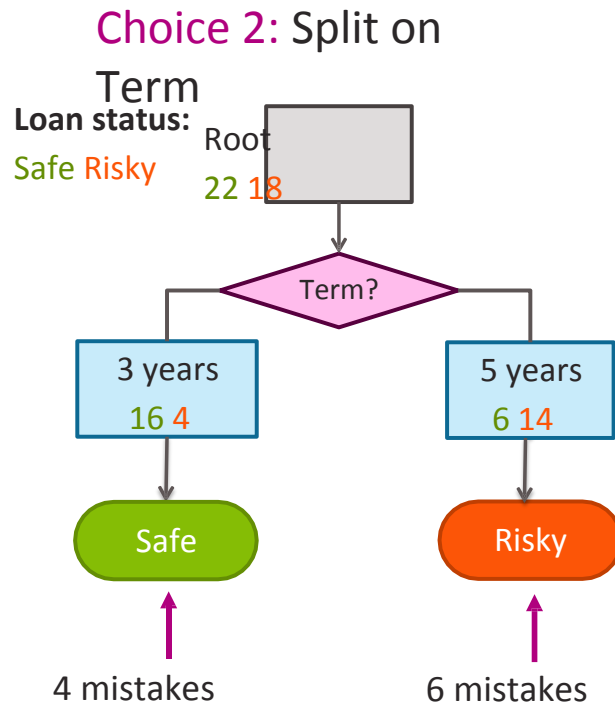
| Tree            | Classification error |
|-----------------|----------------------|
| (root)          | 0.45                 |
| Split on credit | 0.2                  |



## Choice 2: Split on Term?



# Evaluating the split on Term



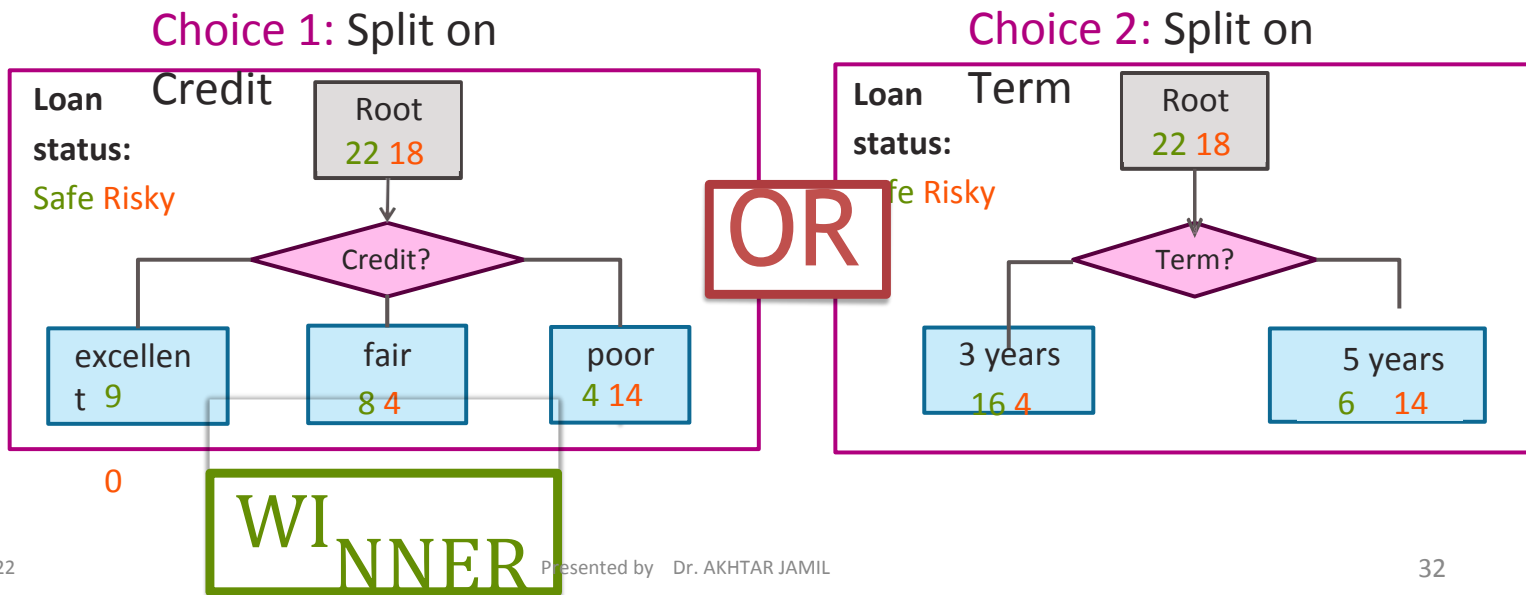
Error =

=

| Tree            | Classification error |
|-----------------|----------------------|
| (root)          | 0.45                 |
| Split on credit | 0.2                  |
| Split on term   | 0.25                 |

# Choice 1 vs Choice 2: Comparing split on Credit vs Term

| Tree               | Classification error |
|--------------------|----------------------|
| (root)             | 0.45                 |
| split on credit    | 0.2                  |
| split on loan term | 0.25                 |



# Feature split selection algorithm

- Given a subset of data  $M$  (a node in a tree)
- For each feature  $h_i(x)$ :
  1. Split data of  $M$  according to feature  $h_i(x)$
  2. Compute classification error of split
- Chose feature  $h^*(x)$  with lowest classification error

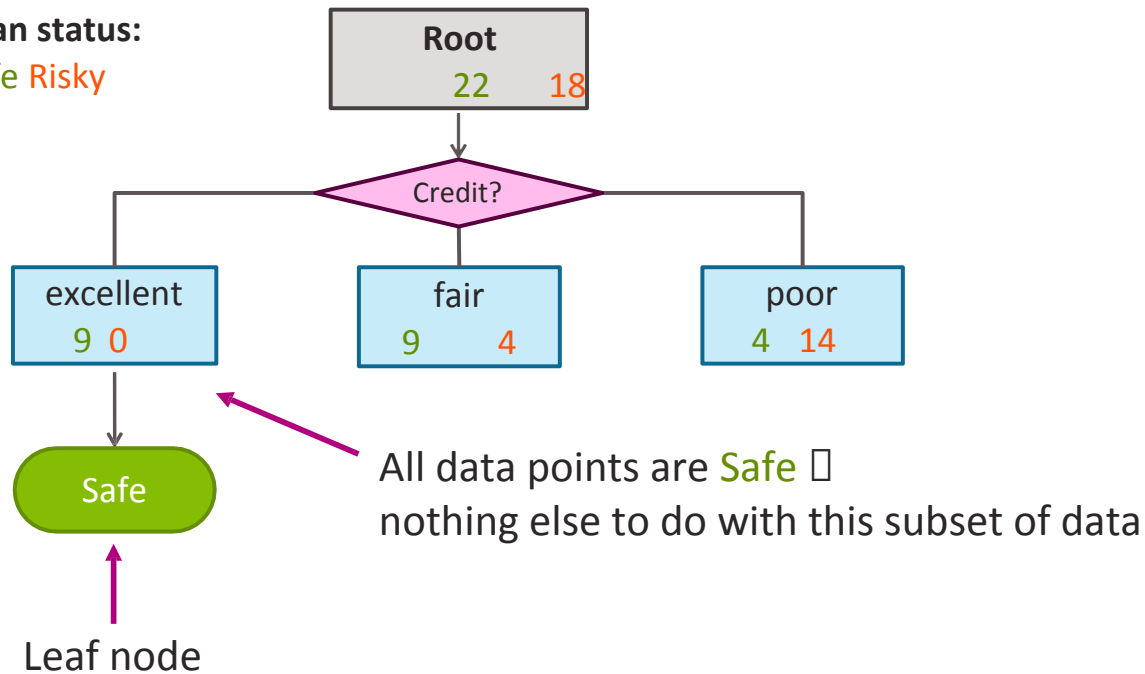
---

# Recursion & Stopping conditions

# We've learned a decision, what next?

Loan status:

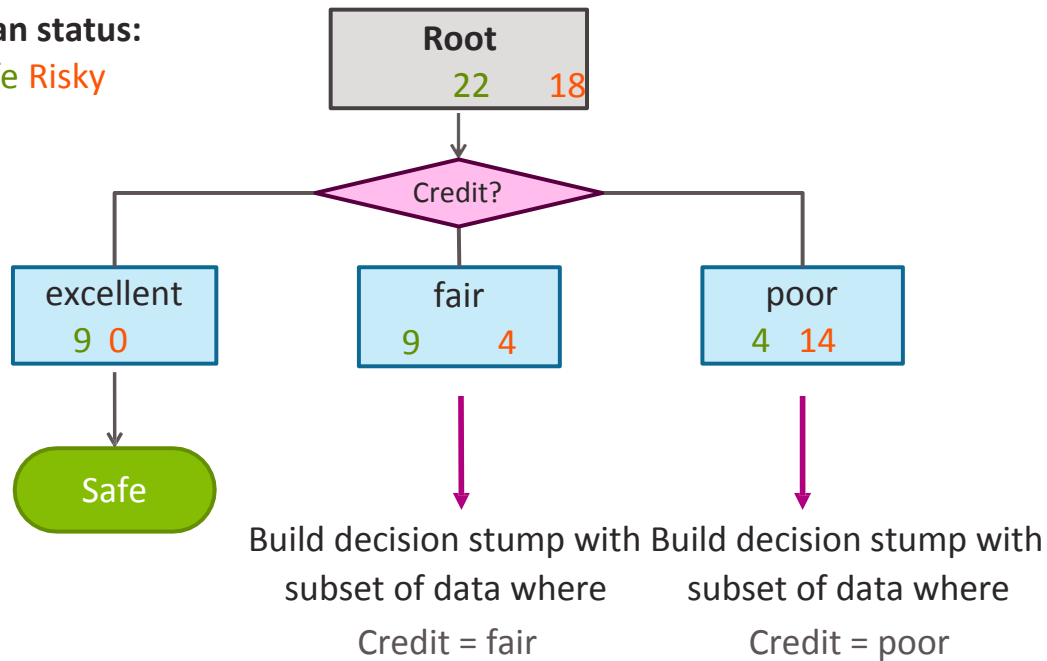
Safe Risky



# Tree learning = Recursive learning

Loan status:

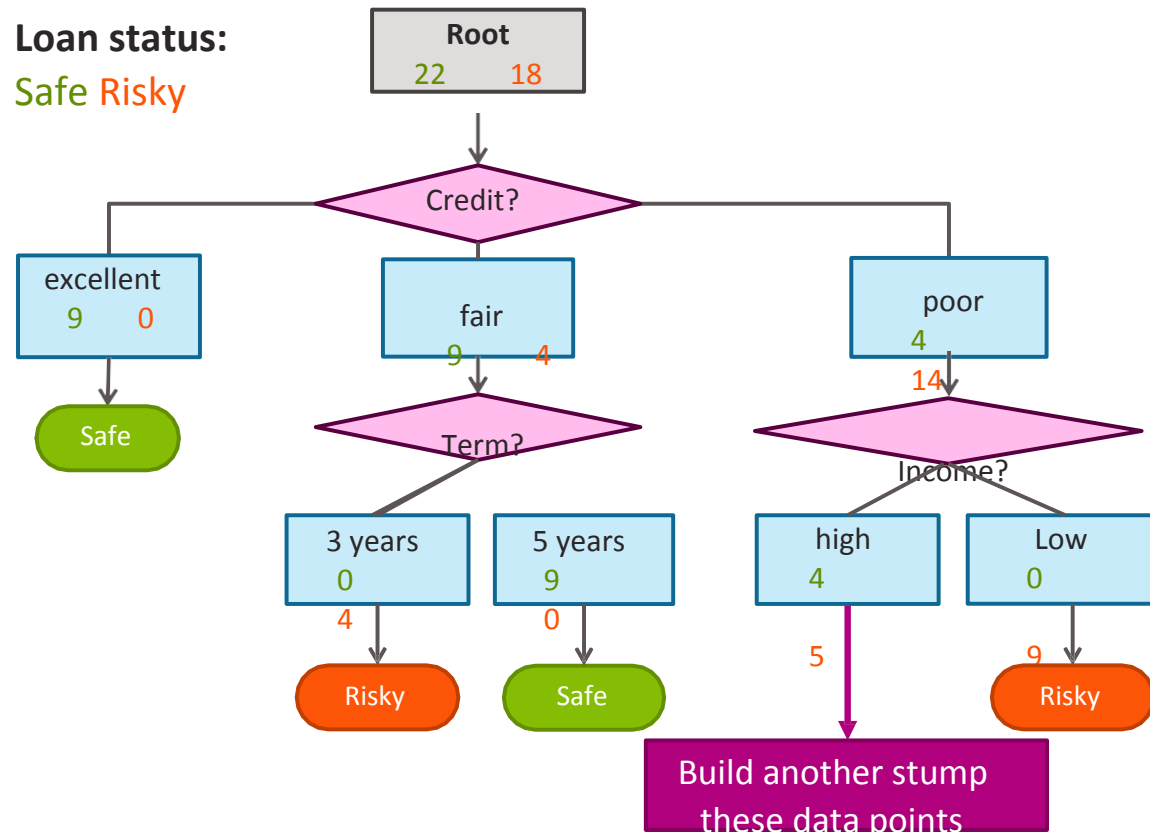
Safe Risky



# Second level

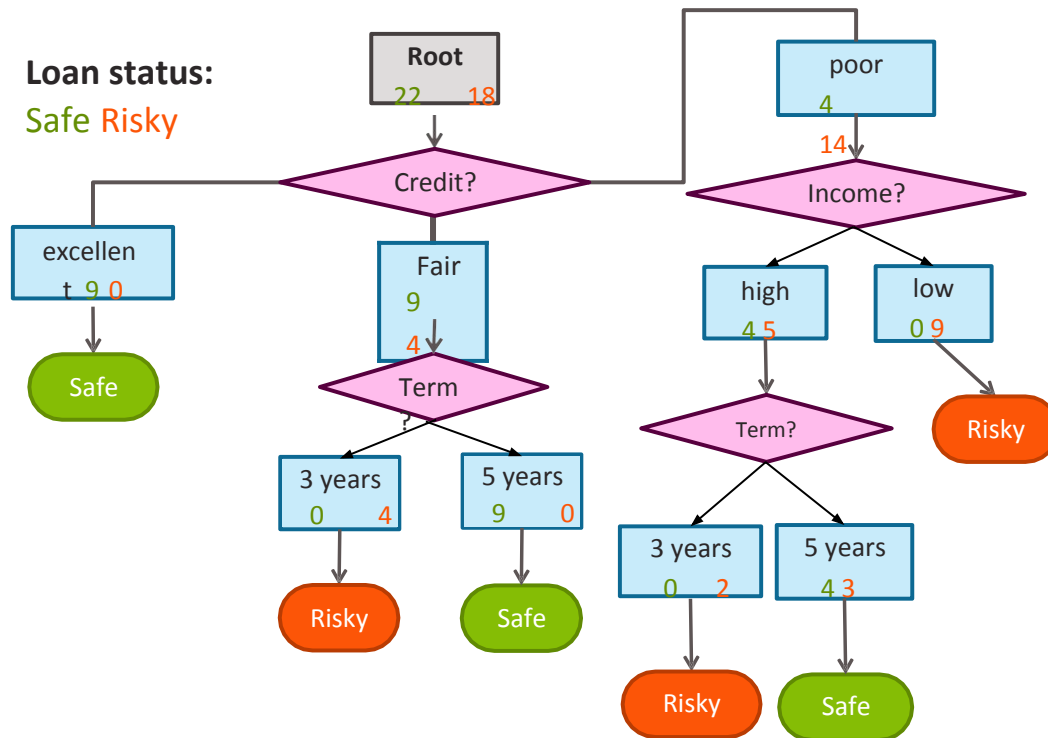
Loan status:

Safe Risky

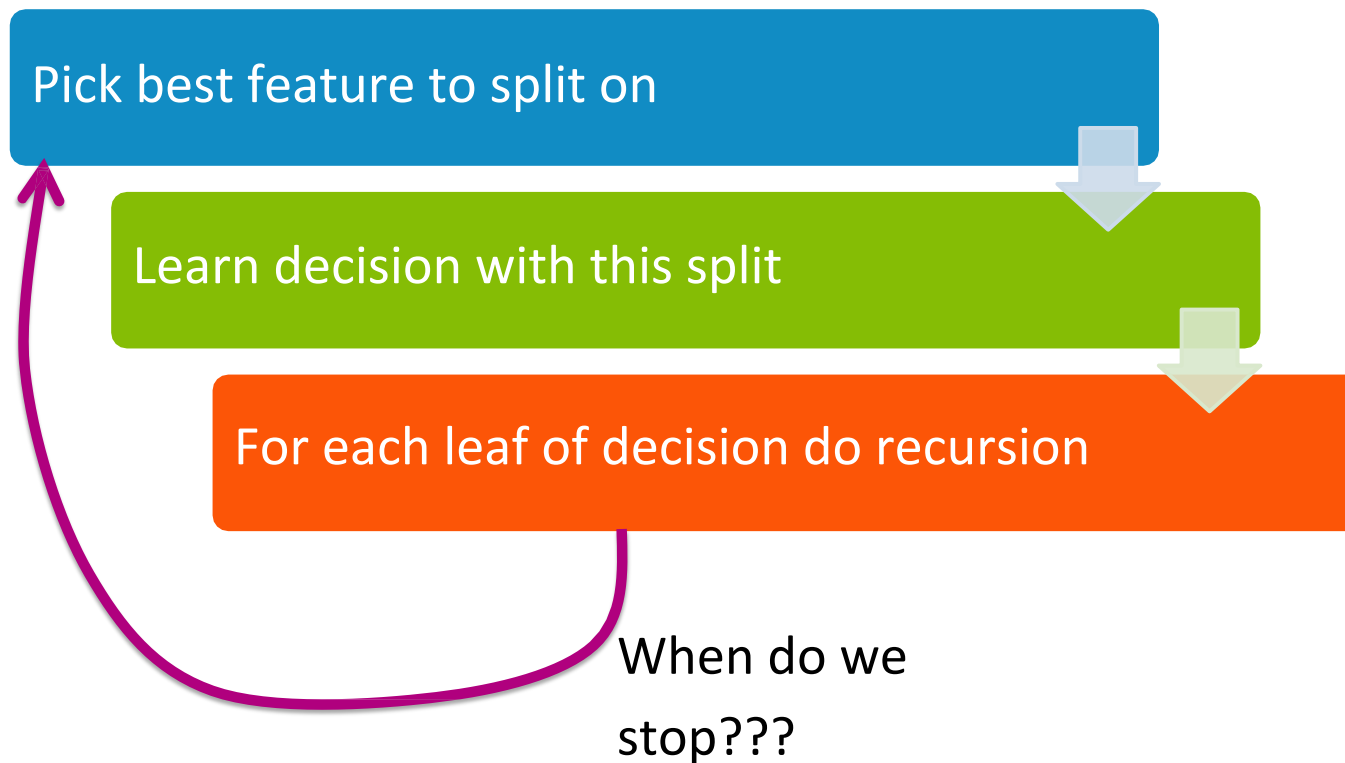




# Final decision tree

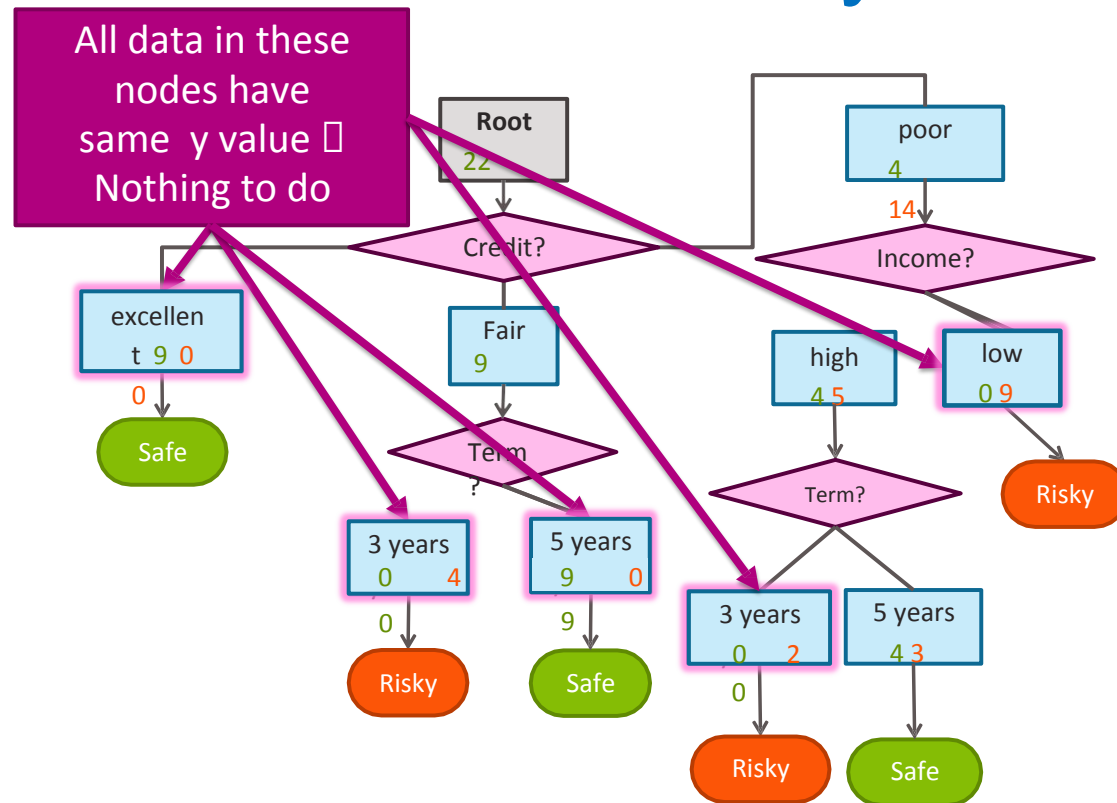


# Simple greedy decision tree learning

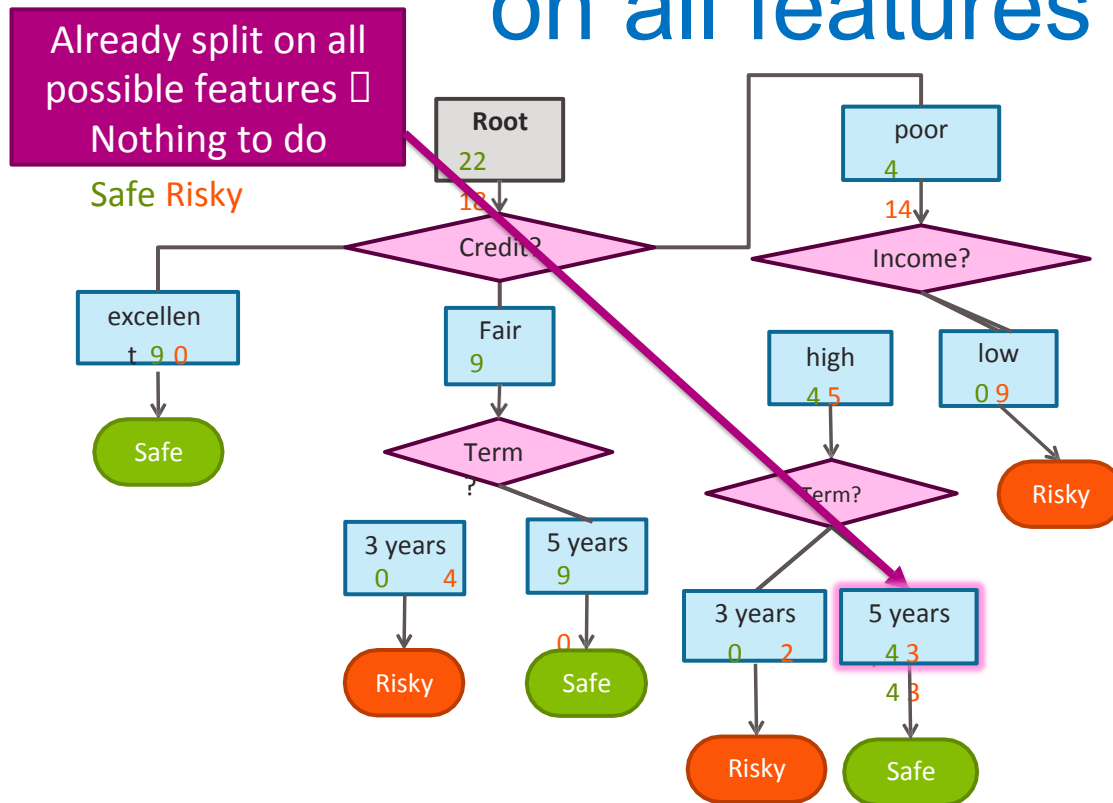


©2021 Carlos Guestrin

# Stopping condition 1: All data agrees on $y$



# Stopping condition 2: Already split on all features



# Greedy decision tree learning

- **Step 1:** Start with an empty tree

- **Step 2:** Select a feature to split data

- For each split of the tree:

- **Step 3:** If nothing more to do, make predictions

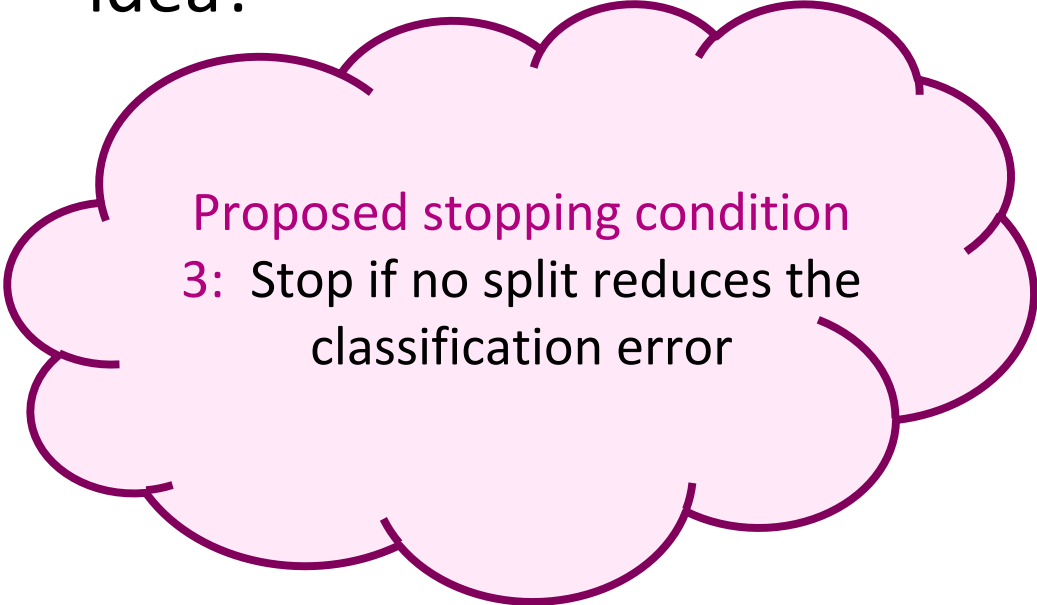
- **Step 4:** Otherwise, go to **Step 2** & continue (recurse) on this split

Pick feature split leading to lowest classification error

Stopping conditions 1 & 2

Recursion

Is this a good  
idea?



Proposed stopping condition  
3: Stop if no split reduces the  
classification error

# References

- Chapter 3, Machine Learning, Tom Mitchell
- <http://cs229.stanford.edu/notes2021spring>

Thank You 😊